

Standalone Packaging, Code Freeze & Final Launch Demo (Damithu / QA)

Document Scope: Executable Packaging, Zero-Dependency Verification, Checksum Validation & Regression Suite

Author / QA Lead: Damithu | **Target Platforms:** Linux / WSL Kali Linux & Windows (Native PowerShell)

Contents

Part 1: Executive Summary & QA Solution Architecture Report	2
Core Engineering Deliverables & Task Breakdown.....	2
2. Repository Directory & Task Structure	3
Part 2: Execution User Manual (Linux, WSL Kali & Windows Native)	4
Option A: Running the Pre-Built Binary on Linux / WSL (Kali or Ubuntu)	4
Option B: Building Fresh from Source & Verifying Packaging on Linux	4
Option C: Running the Full Regression Test Suite.....	4
Option D: Windows Native Compilation (PowerShell / CMD)	5
Part 3: Technical Terms & Concepts Glossary	7

Part 1: Executive Summary & QA Solution Architecture Report

This document details the final stage of the LAN Scanner software development lifecycle, focusing on standalone packaging, binary portability, zero-dependency deployment verification, and release quality assurance (Days 64 to 68). The primary objective of Week 10 was to bundle the CLI application into a self-contained executable that operates on Linux environments without requiring an existing Python installation, while providing full build instructions for Windows PowerShell.

Non-IT Stakeholder Analogy: The Self-Contained Toolbox

Imagine ordering a complex piece of equipment that normally requires hiring a technician with specialized tools to assemble. PyInstaller standalone packaging acts as a pre-assembled, self-contained toolbox. The user does not need to install Python, manage module dependencies, or configure system paths — everything required to run network port scanning, service banner extraction, and offline CVE vulnerability lookups is bundled inside a single executable package.

Core Engineering Deliverables & Task Breakdown

- **Day 64 — Executable Packaging:** Implemented PyInstaller build script with the explicit `--add-data` flag to bundle the local offline CVE lookup database (`nvd_mock_db.json`) directly into the executable package.
- **Days 64–65 — Clean Machine Binary Verification:** Developed an automated 7-check 'clean machine' verification script (`packaging/verify_binary.py`) that tests binary execution on systems without Python dependencies.
- **Days 64–65 — QA Defect Discovery & Remediation:** Identified and permanently fixed two critical release-blocking bugs during binary QA testing: a CSV export crash on mixed result sets, and a path resolution failure where the packaged binary could not locate its bundled CVE data file.
- **Day 68 — Release Checksum & Asset Integrity:** Implemented automated release asset checksum verification (`packaging/verify_release_assets.py`) to prevent release folder corruption or file tampering.
- **Day 68 — Full Regression & Code Freeze:** Executed the full regression test suite, achieving 27/27 test passes (100% pass rate) with 99% code coverage across scanner, CVE, and exporter modules.

2. Repository Directory & Task Structure

Path / File	Task / Milestone	Functional Description & Purpose
main.py	Core CLI	Main application entry point: scans ports, grabs banners, performs offline CVE lookups, exports JSON/CSV.
packaging/build_pyinstaller.sh	Day 64	Shell script building the standalone Linux binary (includes `--add-data` for CVE database bundling).
packaging/verify_binary.py	Days 64–65	Automated clean-machine test suite running 7 verification checks on the packaged binary.
packaging/verify_release_assets.py	Day 68	Release integrity tool that checksum-verifies files in the release folder against SHA256SUMS.txt.
packaging/release/	Day 68	Pre-built Linux executable (`lanscanner-linux-x86_64`), release README, RC1 sign-off, and SHA256SUMS.txt.
qa/reports/pyinstaller_qa_notes.md	Days 64–65	Detailed QA incident report documenting the two real bugs discovered during packaging and their resolution.
tests/test_regression.py	QA Suite	Full regression test suite (27 tests), including dedicated regression tests for the CSV export fix.
scanner/, cve/, exporters/, stress/	Core Engine	Shared Python engine modules supporting socket scanning, CVE matching, data export, and stress testing.

Part 2: Execution User Manual (Linux, WSL Kali & Windows Native)

The LAN Scanner standalone package supports zero-dependency execution on Linux/WSL environments via a pre-built binary, as well as native Windows compilation using PyInstaller in PowerShell or CMD.

Option A: Running the Pre-Built Binary on Linux / WSL (Kali or Ubuntu)

To run the pre-built standalone executable directly without installing Python or third-party packages:

```
cd Week10_QA_Packaging_and_Release
chmod +x packaging/release/lanscanner-linux-x86_64
./packaging/release/lanscanner-linux-x86_64 --target 127.0.0.1 --ports 20-25,80,443 --cve
```

Option B: Building Fresh from Source & Verifying Packaging on Linux

To compile the standalone binary from source and run automated clean-machine verification:

```
python3 -m pip install rich pyinstaller
bash packaging/build_pyinstaller.sh
python3 packaging/verify_binary.py          # Runs 7-point clean-machine
check
python3 packaging/verify_release_assets.py  # Checksum-verifies
packaging/release/
```

Option C: Running the Full Regression Test Suite

To execute the complete regression suite and verify test coverage:

```
python3 -m pip install pytest pytest-cov psutil
python3 -m pytest tests/ -v --cov=scanner --cov=cve --cov=exporters
```



```
# Build Windows executable
py -m pip install pyinstaller rich
py -m PyInstaller --onefile --name lanscanner --add-data
"cve\nvd_mock_db.json;cve" main.py
dist\lanscanner.exe --target 127.0.0.1 --ports 20-25,80,443 --cve

# Run from source without building .exe
py -m pip install rich
py main.py --target 127.0.0.1 --ports 20-25,80,443 --cve
```

Part 3: Technical Terms & Concepts Glossary

PyInstaller

Technical Definition: A utility that freezes (packages) Python applications into stand-alone executables, bundling the Python interpreter, bytecode, and dependencies so end users can run the app without installing Python.

Non-IT Analogy: *Like sealing a fully functional appliance inside a single box so the customer can plug it in immediately without needing to buy extra parts.*

Binary Portability

Technical Definition: The ability of a compiled binary file to run seamlessly across different systems of the same CPU architecture without requiring local libraries or external runtimes.

Non-IT Analogy: *Like a universal USB flash drive that plugs into and works on any standard computer immediately.*

Clean Machine Verification

Technical Definition: A testing process where an executable is executed on a freshly configured system that lacks development tools or local Python runtimes to ensure zero hidden external dependencies.

Non-IT Analogy: *Like test-driving a new car in an isolated desert test track to prove it functions without being connected to factory power cables.*

--add-data Bundling Fix

Technical Definition: A PyInstaller build flag used to explicitly include non-Python data files (such as the offline CVE JSON database) inside the compiled binary bundle.

Non-IT Analogy: *Like ensuring a user manual and reference handbook are packed inside the box alongside the main appliance before shipping.*

SHA256 Checksum Validation

Technical Definition: A cryptographic hash function used to verify that release files have not been corrupted during transit, altered, or tampered with.

Non-IT Analogy: *Like a tamper-evident wax seal on an official document; if the seal is broken or modified, the reader knows it was altered.*

Full Regression Suite

Technical Definition: A comprehensive set of automated tests run before every release to verify that new code changes or packaging updates have not broken existing functionality.

Non-IT Analogy: *Like conducting a full 50-point safety inspection on a vehicle after replacing the engine to ensure headlights and brakes still work perfectly.*